

# 30 Days SQL Micro Course

## Practise Sql Queries 2

### Queries using only:

SQL Execution Order (Remember)

- CREATE DATABASE
- USE DATABASE
- CREATE TABLE
- INSERT DATA
- SELECT
- WHERE
- GROUP BY
- HAVING
- ORDER BY
- LIMIT
- UPDATE
- DELETE
- ALTER
- TRUNCATE
- DROP

Practise link : <https://dbfiddle.uk/vKsjhDAw>

### Step 1: Create Database

Question:

Create a database named CompanyDB.

Syntax

CREATE DATABASE database\_name;

Query

CREATE DATABASE CompanyDB;



1

**CREATE DATABASE** CompanyDB;

**CREATE DATABASE**

---

## Step 2: Use Database

Question:

Select CompanyDB database.

Syntax

USE database\_name;

Query

USE CompanyDB;

(PostgreSQL users usually select database from pgAdmin instead of USE.)

---

## Step 3: Create Table

Question:

Create Employees table.

Syntax

```
CREATE TABLE table_name(  
    column1 datatype,  
    column2 datatype  
);
```

Query

```
CREATE TABLE Employees(  
    Emp_ID INT,  
    Name VARCHAR(50),  
    Department VARCHAR(50),  
    Salary INT,
```

City VARCHAR(50)  
);



```
1 v CREATE TABLE Employees(  
2     Emp_ID INT,  
3     Name VARCHAR(50),  
4     Department VARCHAR(50),  
5     Salary INT,  
6     City VARCHAR(50)  
7 );
```

| CREATE TABLE



```
1 v SELECT *  
2 FROM Employees;
```

| emp_id | name | department | salary | city |
|--------|------|------------|--------|------|
|--------|------|------------|--------|------|

| SELECT 0

#### Step 4: Insert Data

##### Question:

Insert employee records.

##### Syntax

```
INSERT INTO table_name  
VALUES (...);
```

##### Query

```
INSERT INTO Employees VALUES
(1,'Rahul','IT',50000,'Surat'),
(2,'Amit','HR',40000,'Ahmedabad'),
(3,'Priya','IT',60000,'Vadodara'),
(4,'Neha','Sales',45000,'Surat'),
(5,'Raj','HR',55000,'Ahmedabad');
```



```
1  INSERT INTO Employees VALUES
2  (1, 'Rahul', 'IT', 50000, 'Surat'),
3  (2, 'Amit', 'HR', 40000, 'Ahmedabad'),
4  (3, 'Priya', 'IT', 60000, 'Vadodara'),
5  (4, 'Neha', 'Sales', 45000, 'Surat'),
6  (5, 'Raj', 'HR', 55000, 'Ahmedabad');
```

| INSERT 0 5

#### Step 5: View Data

Question:

Show all records.

```
SELECT *
FROM Employees;
```



```
1  SELECT *
2  FROM Employees;
```

| emp_id | name  | department | salary | city      |
|--------|-------|------------|--------|-----------|
| 1      | Rahul | IT         | 50000  | Surat     |
| 2      | Amit  | HR         | 40000  | Ahmedabad |
| 3      | Priya | IT         | 60000  | Vadodara  |
| 4      | Neha  | Sales      | 45000  | Surat     |
| 5      | Raj   | HR         | 55000  | Ahmedabad |

| SELECT 5

---

### Step 6: Show Specific Columns

Question:

Show employee names and salaries.

```
SELECT Name, Salary  
FROM Employees;
```



```
1 ✓ SELECT Name, Salary  
2 FROM Employees;
```

| name  | salary |
|-------|--------|
| Rahul | 50000  |
| Amit  | 40000  |
| Priya | 60000  |
| Neha  | 45000  |
| Raj   | 55000  |

| SELECT 5

---

### Step 7: WHERE

Question:

Show employees whose salary is more than 50000.

```
SELECT *  
FROM Employees  
WHERE Salary > 50000;
```



```
1 v SELECT *
2 FROM Employees
3 WHERE Salary > 50000;
```

| emp_id | name  | department | salary | city      |
|--------|-------|------------|--------|-----------|
| 3      | Priya | IT         | 60000  | Vadodara  |
| 5      | Raj   | HR         | 55000  | Ahmedabad |

| SELECT 2

Question:

Show employees from Surat.

```
SELECT *
FROM Employees
WHERE City='Surat';
```



```
1 v SELECT *
2 FROM Employees
3 WHERE City='Surat';
```

| emp_id | name  | department | salary | city  |
|--------|-------|------------|--------|-------|
| 1      | Rahul | IT         | 50000  | Surat |
| 4      | Neha  | Sales      | 45000  | Surat |

| SELECT 2

Step 8: ORDER BY

Question:

Show employees by highest salary.

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC;
```



```
1 v SELECT *  
2 FROM Employees  
3 ORDER BY Salary DESC;
```

| emp_id | name  | department | salary | city      |
|--------|-------|------------|--------|-----------|
| 3      | Priya | IT         | 60000  | Vadodara  |
| 5      | Raj   | HR         | 55000  | Ahmedabad |
| 1      | Rahul | IT         | 50000  | Surat     |
| 4      | Neha  | Sales      | 45000  | Surat     |
| 2      | Amit  | HR         | 40000  | Ahmedabad |

| SELECT 5

Question:

Sort names alphabetically.

```
SELECT *  
FROM Employees  
ORDER BY Name ASC;
```



```
1 v SELECT *  
2 FROM Employees  
3 ORDER BY Name ASC;
```

| emp_id | name  | department | salary | city      |
|--------|-------|------------|--------|-----------|
| 2      | Amit  | HR         | 40000  | Ahmedabad |
| 4      | Neha  | Sales      | 45000  | Surat     |
| 3      | Priya | IT         | 60000  | Vadodara  |
| 1      | Rahul | IT         | 50000  | Surat     |
| 5      | Raj   | HR         | 55000  | Ahmedabad |

| SELECT 5

### Step 9: LIMIT

Question:

Show first 3 employees.

```
SELECT *  
FROM Employees  
LIMIT 3;
```



```
1  SELECT *  
2  FROM Employees  
3  LIMIT 3;
```

| emp_id | name  | department | salary | city      |
|--------|-------|------------|--------|-----------|
| 1      | Rahul | IT         | 50000  | Surat     |
| 2      | Amit  | HR         | 40000  | Ahmedabad |
| 3      | Priya | IT         | 60000  | Vadodara  |

| SELECT 3

### Step 10: DISTINCT

Question:

Show unique cities.

```
SELECT DISTINCT City  
FROM Employees;
```



```
1  SELECT DISTINCT City  
2  FROM Employees;
```

| city      |
|-----------|
| Vadodara  |
| Surat     |
| Ahmedabad |

| SELECT 3



---

### Step 11: COUNT

Question:

Count employees.

```
SELECT COUNT(*) AS Total_Employees  
FROM Employees;
```

Output:

5



```
1 v SELECT COUNT(*) AS Total_Employees  
2 FROM Employees;
```

| total_employees |
|-----------------|
| 5               |

| SELECT 1

---

### Step 12: SUM

Question:

Find total salary.

```
SELECT SUM(Salary) AS Total_Salary  
FROM Employees;
```



```
1 v SELECT SUM(Salary) AS Total_Salary  
2 FROM Employees;
```

| total_salary |
|--------------|
| 250000       |

| SELECT 1

---

---

### Step 13: AVG

Question:

Find average salary.

```
SELECT AVG(Salary) AS Avg_Salary
FROM Employees;
```



```
1 v SELECT AVG(Salary) AS Avg_Salary
2 FROM Employees;
```

| avg_salary         |
|--------------------|
| 50000.000000000000 |



| SELECT 1

---

### Step 14: MAX

Question:

Find highest salary.

```
SELECT MAX(Salary) AS Highest_Salary
FROM Employees;
```



```
1 v SELECT MAX(Salary) AS Highest_Salary
2 FROM Employees;
```

| highest_salary |
|----------------|
| 60000          |



| SELECT 1

---

### Step 15: MIN

Question:

Find lowest salary.

```
SELECT MIN(Salary) AS Lowest_Salary
FROM Employees;
```



```
1 v SELECT MIN(Salary) AS Lowest_Salary
2 FROM Employees;
```

| lowest_salary |
|---------------|
| 40000         |

| SELECT 1

Step 16: GROUP BY

Question:

Count employees department-wise.

```
SELECT Department,
COUNT(*) AS Employee_Count
FROM Employees
GROUP BY Department;
```

Output:



```
1 v SELECT Department,
2 COUNT(*) AS Employee_Count
3 FROM Employees
4 GROUP BY Department;
```

| department | employee_count |
|------------|----------------|
| Sales      | 1              |
| IT         | 2              |
| HR         | 2              |

| SELECT 3

---

Question:

Average salary department-wise.

```
SELECT Department,  
AVG(Salary) AS Avg_Salary  
FROM Employees  
GROUP BY Department;
```



```
1  v  SELECT Department,  
2    AVG(Salary) AS Avg_Salary  
3    FROM Employees  
4    GROUP BY Department;
```

| department | avg_salary         |
|------------|--------------------|
| Sales      | 45000.000000000000 |
| IT         | 55000.000000000000 |
| HR         | 47500.000000000000 |

| SELECT 3

---

Question:

Total salary department-wise.

```
SELECT Department,  
SUM(Salary) AS Total_Salary  
FROM Employees  
GROUP BY Department;
```



```
1 v SELECT Department,  
2   SUM(Salary) AS Total_Salary  
3   FROM Employees  
4   GROUP BY Department;
```

| department | total_salary |
|------------|--------------|
| Sales      | 45000        |
| IT         | 110000       |
| HR         | 95000        |

| SELECT 3

## Step 17: HAVING

### Question:

Show departments having more than 1 employee.

```
SELECT Department,  
COUNT(*) AS Employee_Count  
FROM Employees  
GROUP BY Department  
HAVING COUNT(*) > 1;
```



```
1 v SELECT Department,  
2   COUNT(*) AS Employee_Count  
3   FROM Employees  
4   GROUP BY Department  
5   HAVING COUNT(*) > 1;
```

| department | employee_count |
|------------|----------------|
| IT         | 2              |
| HR         | 2              |

| SELECT 2

Question:

Show departments whose average salary is above 50000.

```
SELECT Department,  
AVG(Salary)  
FROM Employees  
GROUP BY Department  
HAVING AVG(Salary) > 50000;
```



```
1 v SELECT Department,  
2   AVG(Salary)  
3   FROM Employees  
4   GROUP BY Department  
5   HAVING AVG(Salary) > 50000;
```

| department | avg                |
|------------|--------------------|
| IT         | 55000.000000000000 |

| SELECT 1

---

## Step 18: CASE WHEN

Question:

Categorize salary.

```
SELECT Name,  
Salary,  
CASE  
WHEN Salary >= 50000 THEN 'High Salary'  
ELSE 'Low Salary'  
END AS Salary_Status  
FROM Employees;
```



```
1 v SELECT Name,  
2   Salary,  
3   CASE  
4   WHEN Salary >= 50000 THEN 'High Salary'  
5   ELSE 'Low Salary'  
6   END AS Salary_Status  
7   FROM Employees;
```

| name  | salary | salary_status |
|-------|--------|---------------|
| Rahul | 50000  | High Salary   |
| Amit  | 40000  | Low Salary    |
| Priya | 60000  | High Salary   |
| Neha  | 45000  | Low Salary    |
| Raj   | 55000  | High Salary   |

---

| SELECT 5

---

## Step 19: LIKE

Question:

Names starting with R.

```
SELECT *  
FROM Employees  
WHERE Name LIKE 'R%';
```



```
1 v SELECT *  
2 FROM Employees  
3 WHERE Name LIKE 'R%';
```

| emp_id | name  | department | salary | city      |
|--------|-------|------------|--------|-----------|
| 1      | Rahul | IT         | 50000  | Surat     |
| 5      | Raj   | HR         | 55000  | Ahmedabad |



| SELECT 2

Question:

Names ending with a.

```
SELECT *  
FROM Employees  
WHERE Name LIKE '%a';
```



```
1 v SELECT *  
2 FROM Employees  
3 WHERE Name LIKE '%a';
```

| emp_id | name  | department | salary | city     |
|--------|-------|------------|--------|----------|
| 3      | Priya | IT         | 60000  | Vadodara |
| 4      | Neha  | Sales      | 45000  | Surat    |



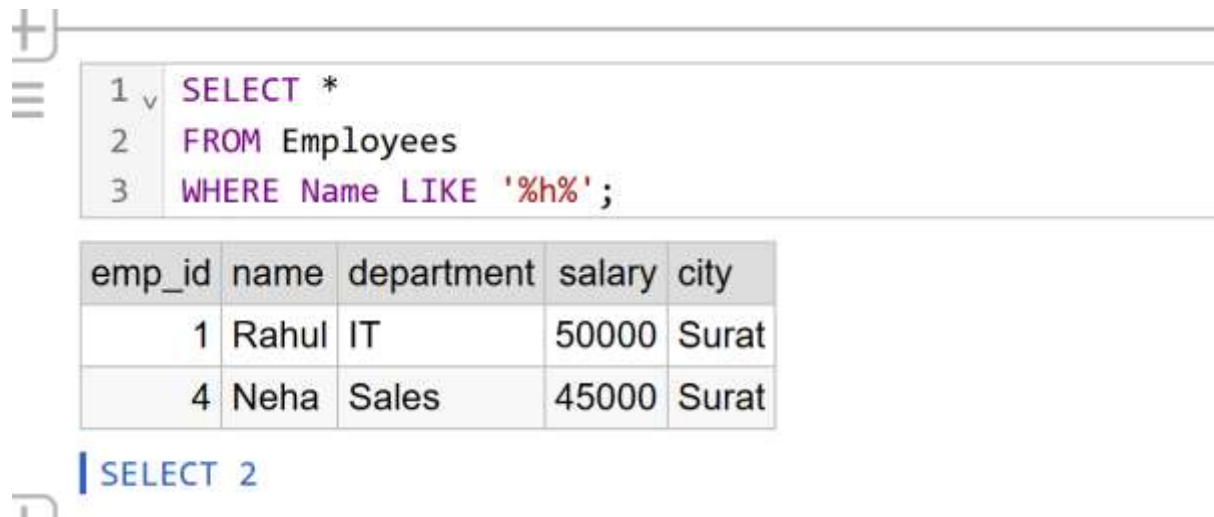
| SELECT 2



Question:

Names containing h.

```
SELECT *  
FROM Employees  
WHERE Name LIKE '%h%';
```



The screenshot shows a SQL IDE interface. On the left, there are icons for a file explorer and a query runner. The main area displays a SQL query with line numbers 1, 2, and 3. Below the query, a table of results is shown with columns: emp\_id, name, department, salary, and city. The results table contains two rows: (1, Rahul, IT, 50000, Surat) and (4, Neha, Sales, 45000, Surat). At the bottom, a blue cursor is positioned over the text 'SELECT 2'.

```
1 SELECT *  
2 FROM Employees  
3 WHERE Name LIKE '%h%';
```

| emp_id | name  | department | salary | city  |
|--------|-------|------------|--------|-------|
| 1      | Rahul | IT         | 50000  | Surat |
| 4      | Neha  | Sales      | 45000  | Surat |

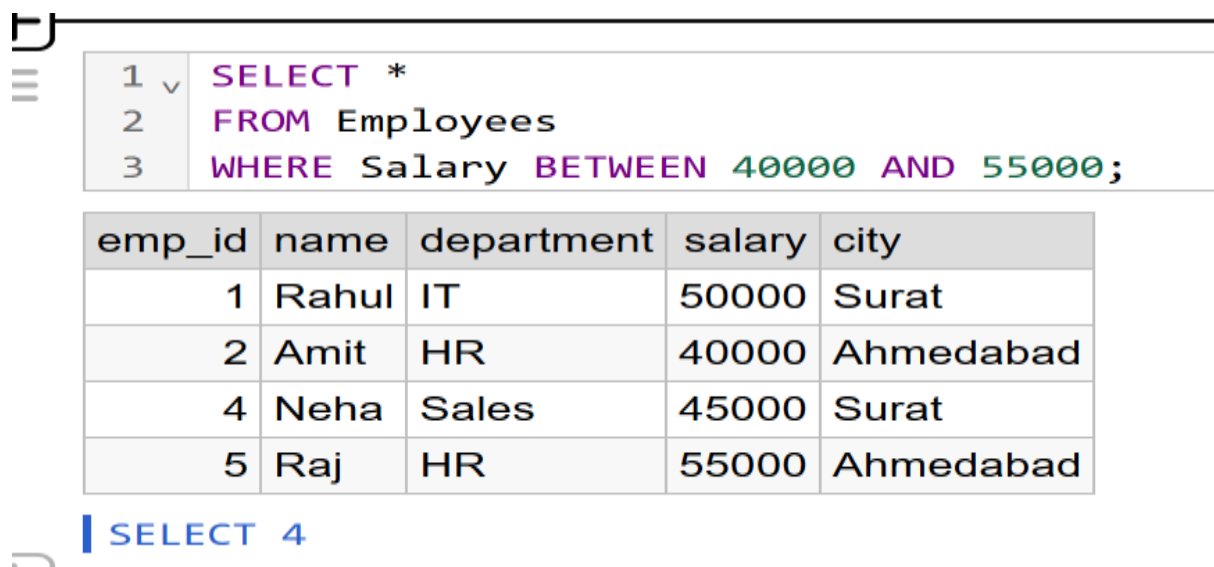
| SELECT 2

Step 20: BETWEEN

Question:

Salary between 40000 and 55000.

```
SELECT *  
FROM Employees  
WHERE Salary BETWEEN 40000 AND 55000;
```



The screenshot shows a SQL IDE interface. On the left, there are icons for a file explorer and a query runner. The main area displays a SQL query with line numbers 1, 2, and 3. Below the query, a table of results is shown with columns: emp\_id, name, department, salary, and city. The results table contains four rows: (1, Rahul, IT, 50000, Surat), (2, Amit, HR, 40000, Ahmedabad), (4, Neha, Sales, 45000, Surat), and (5, Raj, HR, 55000, Ahmedabad). At the bottom, a blue cursor is positioned over the text 'SELECT 4'.

```
1 SELECT *  
2 FROM Employees  
3 WHERE Salary BETWEEN 40000 AND 55000;
```

| emp_id | name  | department | salary | city      |
|--------|-------|------------|--------|-----------|
| 1      | Rahul | IT         | 50000  | Surat     |
| 2      | Amit  | HR         | 40000  | Ahmedabad |
| 4      | Neha  | Sales      | 45000  | Surat     |
| 5      | Raj   | HR         | 55000  | Ahmedabad |

| SELECT 4

---

### Step 21: IN

#### Question:

Employees from Surat and Ahmedabad.

```
SELECT *  
FROM Employees  
WHERE City IN ('Surat','Ahmedabad');
```



```
1 v SELECT *  
2 FROM Employees  
3 WHERE City IN ('Surat','Ahmedabad');
```

| emp_id | name  | department | salary | city      |
|--------|-------|------------|--------|-----------|
| 1      | Rahul | IT         | 50000  | Surat     |
| 2      | Amit  | HR         | 40000  | Ahmedabad |
| 4      | Neha  | Sales      | 45000  | Surat     |
| 5      | Raj   | HR         | 55000  | Ahmedabad |



```
SELECT 4
```

---

### Step 22: Subquery

#### Question:

Employees earning above average salary.

```
SELECT *  
FROM Employees  
WHERE Salary >  
(  
SELECT AVG(Salary)  
FROM Employees  
);
```



```
1 SELECT *
2 FROM Employees
3 WHERE Salary >
4 (
5 SELECT AVG(Salary)
6 FROM Employees
7 );
```

| emp_id | name  | department | salary | city      |
|--------|-------|------------|--------|-----------|
| 3      | Priya | IT         | 60000  | Vadodara  |
| 5      | Raj   | HR         | 55000  | Ahmedabad |

| SELECT 2

#### Step 23: Second Highest Salary

```
SELECT MAX(Salary)
FROM Employees
WHERE Salary <
(
SELECT MAX(Salary)
FROM Employees
);
```



```
1 SELECT MAX(Salary)
2 FROM Employees
3 WHERE Salary <
4 (
5 SELECT MAX(Salary)
6 FROM Employees
7 );
```

| max   |
|-------|
| 55000 |

| SELECT 1

---

### Step 24: UPDATE

Question:

Increase Rahul salary.

```
UPDATE Employees  
SET Salary = 55000  
WHERE Name='Rahul';
```



```
1  UPDATE Employees  
2  SET Salary = 55000  
3  WHERE Name='Rahul';
```

| UPDATE 1

---

### Step 25: DELETE

Question:

Delete Raj.

```
DELETE FROM Employees  
WHERE Name='Raj';
```



```
1  DELETE FROM Employees  
2  WHERE Name='Raj';
```

| DELETE 1

---

### Step 26: ALTER

Question:

Add Email column.

ALTER TABLE Employees  
ADD Email VARCHAR(100);



```
1 v ALTER TABLE Employees
2   ADD Email VARCHAR(100);
```

| ALTER TABLE

Question:

Rename column.

ALTER TABLE Employees  
RENAME COLUMN Name TO Employee\_Name;



```
1 v ALTER TABLE Employees
2   RENAME COLUMN Name TO Employee_Name;
```

| ALTER TABLE

Step 27: TRUNCATE

Question:

Delete all data but keep table.

TRUNCATE TABLE Employees;



```
1 TRUNCATE TABLE Employees;
```

| TRUNCATE TABLE

Step 28: DROP

Question:

Delete table permanently.

DROP TABLE Employees;

```
1  ✓ SELECT pid, datname
2    FROM pg_stat_activity
3    WHERE datname = 'CompanyDB';
```

| pid | datname |
|-----|---------|
|-----|---------|

```
| SELECT 0
```